3.5 Vista de Despliegue

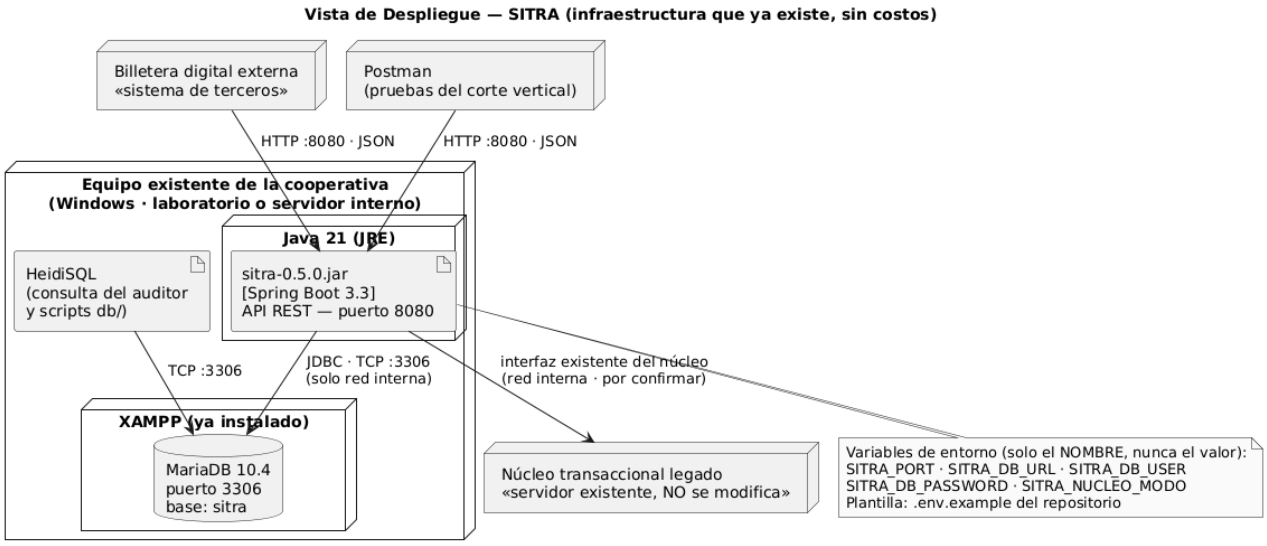


Figura 5. Vista de Despliegue: todo corre en un equipo que la cooperativa ya tiene.

Leyenda

- Caja 3D (nodo): máquina o entorno de ejecución. Hoja con esquina doblada (artefacto): lo que se ejecuta ahí. Cilindro: base de datos.
- Texto de las flechas: protocolo y puerto. La nota lista las variables de entorno sensibles, solo por su nombre.

Lectura en 30 segundos

No se compra ni se instala ninguna plataforma nueva: en un equipo de la cooperativa ya existen XAMPP (con MariaDB y HeidiSQL) y Java. SITRA es un solo archivo JAR que se ejecuta con java -jar y escucha en el puerto 8080; se conecta a MariaDB por el 3306, que solo se usa dentro de la red interna. La billetera entra por el 8080 y el auditor puede consultar la bitácora con HeidiSQL. El núcleo legado sigue en su propio servidor y solo el conector de SITRA le habla.

Puertos y variables de entorno

Elemento	Valor	Dónde se configura
Puerto de la API	8080 (HTTP en laboratorio; HTTPS en producción)	SITRA_PORT
Base de datos	jdbc:mariadb://localhost:3306/sitra	SITRA_DB_URL
Usuario y contraseña de la BD	solo el nombre de la variable; el valor nunca se versiona	SITRA_DB_USER, SITRA_DB_PASSWORD
Modo del conector del núcleo	simulado | sin-respuesta | legado	SITRA_NUCLEO_MODO

Correspondencia con el repositorio: .env.example (nombres sin valores), src/main/resources/application.properties (lee esas variables) y la sección "Ejecutar" del README.md.

SECCIÓN 4: REGISTRO DE DECISIONES DE ARQUITECTURA (ADRs)

Tres decisiones críticas, cada una en docs/adr/ del repositorio con la plantilla obligatoria. Los ADR no se editan: si una decisión cambia, se escribe uno nuevo que la reemplaza (así se hizo con la base de datos en el ADR-002).

ADR	Decisión	Atributo / restricción que la obliga
ADR-001	N-Capas con un conector intercambiable para el núcleo	AC-03 Mantenibilidad; sin ambiente de pruebas del núcleo; cero personal técnico
ADR-002	MariaDB del XAMPP existente en lugar de PostgreSQL	AC-01 integridad; RA-02 cero presupuesto; RA-03 cero personal
ADR-003	La trazabilidad la garantiza la base de datos (triggers) y los rechazados siempre se guardan	AC-01 Seguridad (responsabilidad, no repudio); exigencia del auditor

ADR-001: Patrón N-Capas con un conector intercambiable para el núcleo legado

Estatus

Aceptada

Fecha

2026-10-24

Contexto

SITRA debe exponer consulta de saldo y transferencias de un núcleo legado de 20 años, sin documentación, que **no se puede modificar** y del que **no existe ambiente de pruebas**. La cooperativa no tiene **personal técnico**: quien herede el sistema debe entenderlo sin ayuda.

Atributos en juego: AC-03 Mantenibilidad (modularidad y capacidad de ser probado) y AC-02 Fiabilidad. Restricciones: núcleo intocable, sin ambiente de pruebas, cero personal técnico, cero presupuesto.

Alternativas consideradas:

- **MVC monolítico**: el controlador hablaría directo con el núcleo y la base de datos.
- **Hexagonal (puertos y adaptadores)**: máxima independencia, pero más clases, interfaces y conceptos.
- **Microservicios**: varios servicios que operar y monitorear.
- **N-Capas**: seguridad → presentación → negocio → datos.

Decisión

Decidimos usar **N-Capas** (seguridad, presentación, negocio y datos) y que el núcleo se use **solo a través de la interfaz `NucleoConector`**, con dos implementaciones que se cambian por configuración (sitra.nucleo.modo): NucleoSimuladoConector para pruebas y NucleoLegadoConector para producción, porque AC-03 exige poder probar todo sin el núcleo real y la restricción de cero personal técnico exige la estructura más estándar posible.

Se descarta MVC porque no deja dónde poner el simulador sin tocar código de negocio. Se descarta Hexagonal completo porque su costo en clases y curva de aprendizaje contradice la restricción de cero personal; de él solo se toma la interfaz del conector. Se descarta Microservicios porque exige personal y presupuesto que no existen.

Consecuencias / Trade-offs

- **Positivas:** cualquier desarrollador Java reconoce la estructura (controller, service, repository, model); el servicio no sabe si habla con el núcleo real o con el simulador; cambiar la integración toca una sola clase y un valor de configuración.
- **Negativas:** más código que un MVC simple (servicios y repositorios separados). El dominio no queda totalmente aislado de Spring, como sí lo haría en Hexagonal.

- **Deuda técnica asumida:** NucleoLegadoConector está vacío porque el protocolo del núcleo está por confirmar con el cliente. **Condición para revisarla:** cuando el cliente confirme cómo expone sus datos el núcleo (archivo, base de datos o servicio), se implementa solo esa clase.

ADR-002: MariaDB (la del XAMPP existente) en lugar de PostgreSQL

Estatus

Aceptada — reemplaza la elección de PostgreSQL 16 de la Sección 2.2 del DAS v0.4.

Fecha

2026-10-24

Contexto

En el DAS v0.4 se eligió PostgreSQL 16. En la sustentación de la Clase 4 el modelo se probó en vivo en los equipos disponibles, que tienen **XAMPP con MariaDB y HeidiSQL** ya instalados. La restricción RA-02 dice que **solo se trabaja con lo que se tiene** y la RA-03 que **no hay personal técnico** para instalar y mantener otro motor.

Atributos en juego: AC-01 Seguridad (integridad de la bitácora), AC-03 Mantenibilidad. Restricciones: cero presupuesto, cero personal técnico, usar la infraestructura existente.

Alternativas consideradas:

- **PostgreSQL 16**: muy completo, pero hay que instalarlo y administrarlo aparte.
- **MariaDB 10.4+ del XAMPP**: ya instalado, gratuito, con HeidiSQL para operarlo.
- **H2 en memoria**: no sirve como evidencia porque se borra al apagar.

Decisión

Decidimos usar **MariaDB 10.4+ incluida en XAMPP**, porque cumple todo lo que exige AC-01 — llaves foráneas, CHECK (MariaDB los aplica desde la 10.2), UNIQUE, transacciones InnoDB y triggers que bloquean la bitácora— y porque ya está instalada, cumpliendo RA-02 y RA-03.

Se descarta PostgreSQL porque, aunque es técnicamente equivalente para este caso, exige instalar y mantener un motor nuevo sin personal que lo haga. Se descarta H2 porque no conserva la evidencia.

Consecuencias / Trade-offs

- **Positivas:** cero instalación; el mismo motor sirve para desarrollo, pruebas y la demostración en clase; HeidiSQL permite al auditor consultar la bitácora sin programar.
- **Negativas:** se pierden funciones propias de PostgreSQL (índices parciales, pgcrypto). Se reemplazan: el índice parcial por uno normal y la huella por SHA2() de MariaDB. Los scripts PostgreSQL de la Clase 4 quedan en el historial de Git, no en la versión actual.
- **Deuda técnica asumida:** XAMPP trae MariaDB 10.4, una versión sin soporte del fabricante. **Condición para revisarla:** si el sistema pasa a producción real, actualizar a una versión con soporte (10.11 LTS); los scripts no cambian.

ADR-003: La trazabilidad la garantiza la base de datos, no solo el código

Estatus

Aceptada

Fecha

2026-10-24

Contexto

La trazabilidad es el atributo crítico del caso (AC-01 Seguridad: responsabilidad y no repudio): el auditor la exige y de ella depende la licencia de operación. En la Clase 4 se comprobó que, si la bitácora la escribe solo la aplicación, **una transferencia insertada directamente por SQL no deja rastro**. Además el auditor pidió registrar **también los intentos rechazados**.

Alternativas consideradas:

- **Solo la aplicación escribe la bitácora**: simple, pero cualquier acceso directo a la base de datos (HeidiSQL, otro programa) se salta el registro.
- **Servicio de auditoría en la nube**: con costo recurrente (prohibido por RA-02).
- **Triggers en la base de datos + registro de rechazos desde el servicio**.

Decisión

Decidimos que **la base de datos garantice la bitácora con triggers**: toda transferencia que se inserte o cambie deja su fila automáticamente; la bitácora no se puede modificar ni borrar; y cada fila guarda una huella SHA-256 encadenada con la anterior. Los **intentos rechazados** se guardan así: si la base de datos o la validación impiden crear la transferencia, el servicio escribe la fila **RECHAZADA** en la bitácora en una transacción propia; si el núcleo la rechaza (saldo insuficiente), la transferencia se guarda con **estado RECHAZADA**. Todo con funciones gratuitas de MariaDB.

Consecuencias / Trade-offs

- **Positivas:** ninguna transferencia queda sin evidencia, sin importar por dónde entre; los rechazados nunca se pierden; la regla no depende de que un programador "se acuerde".
- **Negativas:** parte de la lógica vive en la base de datos (triggers y un procedimiento), que es menos visible que el código Java. Cada inserción hace una escritura extra en la bitácora.
- **Deuda técnica asumida:** el procedimiento registrar_transferencia repite en SQL la regla del TransferenciaService, para poder operar y demostrar desde HeidiSQL. **Condición para revisarla:** si las dos versiones empiezan a diferir, se deja una sola y la otra la llama.

SECCIÓN 5: CORTE VERTICAL — ARRANQUE

5.1 Repositorio Git

```
sitra/
├── docs/            DAS.docx, DAS.pdf, adr/ (3 ADR), diagramas/ (.puml + .png)
├── db/
│   ├── migrations/  V1 tablas · V2 triggers de bitácora · V3 rechazados · V4 IP de origen
│   ├── datos_prueba.sql   billetera, auditor, asociado, cuenta y saldo
│   ├── instalar_todo.sql  todo lo anterior en un solo archivo (F9 en HeidiSQL)
│   └── pruebas/demo_sustentacion.sql
├── src/main/java/co/coop/sitra/
│   ├── controller/   TransferenciaController, AuditoriaController
│   ├── service/      TransferenciaService, AuditoriaService, IdentificacionService
│   ├── repository/   repositorios Spring Data JPA
│   ├── model/        entidades JPA (= tablas)
│   ├── integration/  NucleoConector + simulador + conector legado
│   ├── dto/          cuerpos JSON de entrada y salida
│   └── exception/    errores → códigos HTTP
├── src/main/resources/application.properties
├── postman/coleccion-corte-vertical.json
├── evidencias/      una carpeta por clase
├── .env.example  ·  .gitignore  ·  pom.xml  ·  README.md
```

Política de commits: commits pequeños con formato Conventional Commits (feat:, fix:, docs:, test:, chore:). El repositorio lleva 10 commits al cierre de la Clase 4.

5.2 Script SQL DDL / Migraciones (MariaDB)

Cuatro scripts en orden. V1 crea las tablas; V2 los triggers de la bitácora; V3 el procedimiento que registra también los rechazados; V4 es el ALTER que agrega la IP de origen a la transacción (cambio pedido en la Clase 4). db/instalar_todo.sql los ejecuta todos de una vez. A continuación, V1 completo:

```
-- ======================================================================
-- SITRA — V1: tablas, llaves, CHECK, UNIQUE e índices
-- Motor: MariaDB 10.4+ (XAMPP). Ejecutar en orden: V1, V2, V3, V4.
-- ======================================================================

-- Quién usa SITRA: la billetera externa y el auditor
CREATE TABLE cliente_api (
    id           BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nombre       VARCHAR(120)  NOT NULL,
    client_id    VARCHAR(80)   NOT NULL UNIQUE,
    secret_hash  VARCHAR(255)  NOT NULL,
    rol          VARCHAR(20)   NOT NULL DEFAULT 'BILLETERA',
    activo       BOOLEAN       NOT NULL DEFAULT TRUE,                -- borrado lógico
    created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_cliente_rol CHECK (rol IN ('BILLETERA', 'AUDITOR'))
) ENGINE = InnoDB;

-- Asociado de la cooperativa
CREATE TABLE asociado (
    id          BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    documento   VARCHAR(30)   NOT NULL UNIQUE,
    nombre      VARCHAR(120)  NOT NULL,
    activo      BOOLEAN       NOT NULL DEFAULT TRUE,                -- borrado lógico
    created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;

-- Cuenta (lista blanca de cuentas expuestas). NO guarda saldo: el saldo vive en el núcleo.
CREATE TABLE cuenta_autorizada (
    id             BIGINT       NOT NULL AUTO_INCREMENT PRIMARY KEY,
    asociado_id    BIGINT       NOT NULL,
    numero_cuenta  VARCHAR(30)  NOT NULL UNIQUE,
    tipo           VARCHAR(20)  NOT NULL,
    activo         BOOLEAN      NOT NULL DEFAULT TRUE,                -- borrado lógico
    created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cuenta_asociado FOREIGN KEY (asociado_id) REFERENCES asociado(id),
    CONSTRAINT ck_cuenta_tipo CHECK (tipo IN ('AHORROS', 'CORRIENTE'))
```

```
) ENGINE = InnoDB;

-- Transacción (transferencia)
CREATE TABLE transferencia (
    id                  BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL,
    cuenta_origen_id    BIGINT         NOT NULL,
    cuenta_destino      VARCHAR(30)    NOT NULL,
    monto               DECIMAL(14,2)  NOT NULL,
    estado              VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE',
    referencia_externa  VARCHAR(80)    NOT NULL,                    -- código de operación
    comprobante_nucleo  VARCHAR(80)    NULL,
    motivo_rechazo      VARCHAR(255)   NULL,                        -- Paso 4: por qué se rechazó
    activo              BOOLEAN        NOT NULL DEFAULT TRUE,        -- borrado lógico
    created_at          DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at          DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transf_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_transf_cuenta  FOREIGN KEY (cuenta_origen_id) REFERENCES cuenta_autorizada(id),
    CONSTRAINT uq_transf_idempotencia UNIQUE (referencia_externa),    -- llave de idempotencia
    CONSTRAINT ck_transf_monto  CHECK (monto > 0),                    -- CHECK de monto > 0
    CONSTRAINT ck_transf_estado CHECK (estado IN ('PENDIENTE','CONFIRMADA','RECHAZADA','FALLIDA','ANULADA'))
) ENGINE = InnoDB;

-- Bitácora de auditoría (obligatoria: la trazabilidad es el atributo crítico).
-- Guarda también los intentos RECHAZADOS: código, monto intentado y motivo.
-- monto NO tiene CHECK: debe poder guardar lo que se intentó, aunque sea inválido (ej. -50.000).
CREATE TABLE registro_auditoria (
    id                  BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL,                    -- quién
    transferencia_id    BIGINT         NULL,                        -- vacío si la transferencia no se pudo
crear
    numero_cuenta       VARCHAR(30)    NOT NULL,                    -- sobre qué cuenta
    operacion           VARCHAR(30)    NOT NULL,                    -- qué
    resultado           VARCHAR(20)    NOT NULL,                    -- EXITOSA / RECHAZADA / FALLIDA
    referencia_externa  VARCHAR(80)    NULL,                        -- código de operación intentado
    monto               DECIMAL(14,2)  NULL,                        -- monto intentado
    motivo              VARCHAR(255)   NULL,                        -- por qué se rechazó
    ip_origen           VARCHAR(45)    NOT NULL DEFAULT 'local',
    ocurrido_en         DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,  -- cuándo
    hash_anterior       CHAR(64)       NULL,
    hash_actual         CHAR(64)       NULL UNIQUE,
    CONSTRAINT fk_audit_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_audit_transf  FOREIGN KEY (transferencia_id) REFERENCES transferencia(id),
    CONSTRAINT ck_audit_operacion CHECK (operacion IN
('CONSULTA_SALDO','TRANSFERENCIA','ANULACION','CONSULTA_AUDITORIA')),
    CONSTRAINT ck_audit_resultado CHECK (resultado IN ('EXITOSA','RECHAZADA','FALLIDA'))
) ENGINE = InnoDB;

-- Simulador del núcleo legado: aquí vive el SALDO (SITRA no guarda saldos)
CREATE TABLE nucleo_simulado_cuenta (
    numero_cuenta  VARCHAR(30)    NOT NULL PRIMARY KEY,
    saldo          DECIMAL(14,2)  NOT NULL,
    CONSTRAINT ck_nucleo_saldo CHECK (saldo >= 0)
) ENGINE = InnoDB;

-- Índices justificados
CREATE INDEX idx_transferencia_cuenta   ON transferencia (cuenta_origen_id);
CREATE INDEX idx_auditoria_cuenta_fecha ON registro_auditoria (numero_cuenta, ocurrido_en);
```

db/migrations/V4__ip_origen_en_transferencia.sql (extracto)

```
ALTER TABLE transferencia
    ADD COLUMN ip_origen VARCHAR(45) NULL AFTER referencia_externa;
```

Cumplimiento de los requisitos evaluables

Requisito de la plantilla	Dónde se cumple
PK en toda tabla	id BIGINT AUTO_INCREMENT PRIMARY KEY en las 6 tablas (nucleo_simulado_cuenta usa numero_cuenta).
FK con REFERENCES	cuenta_autorizada → asociado; transferencia → cliente_api y cuenta_autorizada; registro_auditoria → cliente_api y transferencia.

Requisito de la plantilla	Dónde se cumple
Al menos una CHECK	monto > 0 (ck_transf_monto), estado, rol, tipo de cuenta, operacion, resultado y saldo ≥ 0.
Al menos una UNIQUE	referencia_externa = llave de idempotencia (uq_transf_idempotencia); client_id, documento, numero_cuenta, hash_actual.
Borrado lógico	Campo activo en asociado, cliente_api, cuenta_autorizada y transferencia. La bitácora no lo tiene a propósito: es evidencia.
Índices justificados	idx_transferencia_cuenta (historial por cuenta) e idx_auditoria_cuenta_fecha (HU-03: el auditor consulta por cuenta y fechas).

Verificación ejecutada (MariaDB, db/pruebas/demo_sustentacion.sql)

```
Paso 1  transferencia 300.000  ->  bitácora: TRANSFERENCIA · EXITOSA (la escribió el trigger)
Paso 2  mismo código           ->  ERROR 1062 Duplicate entry ... 'uq_transf_idempotencia'
        monto -50.000          ->  ERROR 4025 CONSTRAINT 'ck_transf_monto' failed
Paso 4  CALL con código repetido -> RECHAZADA · queda en la bitácora con su motivo
        CALL con -50.000          -> RECHAZADA · queda en la bitácora con su motivo
        CALL con 5.000.000        -> RECHAZADA · transacción con estado RECHAZADA + bitácora
UPDATE / DELETE sobre registro_auditoria -> ERROR 1644: no se puede modificar / borrar
Paso 5 verificación de la cadena       -> cadena OK · huella OK en todas las filas
```

5.3 Arranque del corte vertical (Spring Boot)

Estado al cierre de la Clase 5: la estructura por capas completa y la entidad principal (Transferencia) con Crear, Listar, Consultar por id y Anular (borrado lógico). La Clase 6 agrega la actualización (PUT) y reemplaza la identificación por encabezado por un token JWT.

Método y ruta	Qué hace	Respuestas
POST /api/v1/transferencias	Registra la transferencia (HU-01) pasando por el núcleo simulado	201 · 400 · 401 · 404 · 409 · 422 · 503
GET /api/v1/transferencias	Lista las transferencias activas	200 · 401
GET /api/v1/transferencias/{id}	Consulta una transferencia	200 · 401 · 404
DELETE /api/v1/transferencias/{id}	Anula (borrado lógico); la bitácora registra ANULACION	200 · 401 · 404
GET /api/v1/auditoria?cuenta=	Bitácora para el rol AUDITOR (HU-03)	200 · 401 · 403

Cómo se ejecuta

- XAMPP Control Panel → Start en MySQL.
- HeidiSQL → abrir db/instalar_todo.sql → F9 (crea la base sitra con datos de prueba).
- VS Code → abrir la carpeta del repositorio → Run sobre SitraApplication.java (o mvn spring-boot:run).
- Postman → importar postman/coleccion-corte-vertical.json → ejecutar las peticiones en orden.

Pendiente

Sección	Contenido	Clase
5.3	CRUD completo (PUT), seguridad con JWT + BCrypt, consulta de saldo (HU-02) y filtro por fechas y verificación de integridad en la API (HU-03; hoy se hacen en SQL, Paso 5 de la demo).	Clase 6 — 07/11
5.4 y 6	Pruebas automatizadas, análisis de trade-offs y riesgos.	Clase 7 — 14/11